

Esame di: **Fondamenti di Robotica**

Consegna: **24 aprile 2026**

Gruppo: **PJ24-16-FR**

Studenti: **Zuccaro Salvatore, 0750527** (Referente)

Baccarella Davide, 0753412

Saitta Roberto, 0758836

Risultato:

(a cura del docente)

Commenti generali alla soluzione *(a cura del docente):*

Linee Guida e Istruzioni per la Consegna

Ogni tema progettuale si propone di far sperimentare allo studente, sia pure in modo semplificato, i diversi momenti di analisi e progettazione, sottesi alla soluzione di un problema di controllo in robotica; ciò avviene, per ovvie ragioni, attraverso uno studio teorico e solo simulato, come è tipico in un corso di fondamenti di robotica, ma rappresenta comunque un momento fondamentale che precede qualsiasi realizzazione completa e sperimentale.

Il modello $\text{\LaTeX}$ include, nella cartella **support**, la classe Matlab **afView**, progettata per creare figure vettoriali con annotazioni leggibili. Il file **tutorial.m**, presente nella stessa cartella, fornisce esempi pratici per la generazione dei grafici. Gli schemi Simulink possono essere esportati utilizzando il comando **afView.SimulinkToPNG(fileName)**, specificando **fileName** come nome del file con estensione **.png**.

La consegna del progetto deve includere il PDF dell'elaborato generato con il presente modello $\text{\LaTeX}$ e un file ZIP contenente:

- i file sorgenti $\text{\LaTeX}$ del PDF (**main.tex**, **robot-theme.tex**, **1*.tex**, **2A.tex**);
- una cartella **img** contenente le immagini e tutti i grafici inseriti nel testo, che devono essere riportati in formato vettoriale e contenere annotazioni adeguate e leggibili;
- una cartella **support** contenente (1) gli script (estensione **.m**) necessari per inizializzare e lanciare la simulazione e (2) i due diagrammi Simulink (estensione **.slx**) di tutti i sottosistemi, correttamente funzionanti e tali da riprodurre i risultati riportati nel documento descrittivo.

1 Parte Controllo Non Lineare

Un sistema dotato di ingresso di controllo u è caratterizzato dal modello dinamico non lineare

$$\begin{cases} \dot{x}_1 &= x_1(x_1^2 + x_2^2 - 4) - x_2 - x_1^3 \\ \dot{x}_2 &= x_2(x_1^2 + x_2^2 - 4) + x_1 + x_3 \\ \dot{x}_3 &= x_1^2 - x_2^2 - x_4 \\ \dot{x}_4 &= -x_1^2 + 3x_2^2 + u(t) \end{cases} \quad (1.1)$$

Si chiede di rispondere ai seguenti quesiti:

- (a) Considerando, per il momento, solo le prime due equazioni del modello dinamico, in cui la variabile x_3 agisce come un ingresso virtuale, e assumendo che tale variabile sia costante, $x_3 = \bar{x}_3$, determinare gli equilibri $(\bar{x}_1, \bar{x}_2, \bar{x}_3)$ del modello dinamico formato dalle prime due equazioni.

Soluzione:

Si consideri il seguente sottosistema:

$$\begin{cases} \dot{x}_1 = x_1(x_1^2 + x_2^2 - 4) - x_2 - x_1^3 \\ \dot{x}_2 = x_2(x_1^2 + x_2^2 - 4) + x_1 + x_3 \end{cases} \quad (1.a.1)$$

Per trovare i punti di equilibrio si pone l'ingresso costante, in questo caso x_3 , e le variabili di stato $\dot{x}_1$ e $\dot{x}_2$, identicamente nulle in quanto, per trovare i punti di equilibrio, è necessario individuare i punti in cui la derivata rispetto al tempo non varia:

$$\begin{cases} 0 = x_1(x_1^2 + x_2^2 - 4) - x_2 - x_1^3 \\ 0 = x_2(x_1^2 + x_2^2 - 4) + x_1 \end{cases}$$

Sicuramente l'origine è un punto di equilibrio, si procede quindi imponendo $x_1, x_2 \neq 0$. La seconda equazione si annulla solo quando:

$$x_2(x_1^2 + x_2^2 - 4) = -x_1$$

Di conseguenza, la quantità dentro la parentesi deve essere pari a $-x_1/x_2$. Sostituendola alla prima equazione si ha:

$$0 = -\frac{x_1^2}{x_2} - x_2 - x_1^3$$

Moltiplicando ogni membro per x_2 si ottiene:

$$0 = -x_1^2 - x_2^2 - x_1^3 x_2$$

Risolvendo per x_2 , si ha:

$$x_2 = \frac{-x_1^3 \pm \sqrt{(x_1^3)^2 - 4x_1^2}}{2} \quad (1.a.2)$$

Di conseguenza i punti di equilibrio sono:

$$P_1 = (0, 0), P_{2,3} = \left(x_1, \frac{-x_1^3 \pm \sqrt{(x_1^3)^2 - 4x_1^2}}{2} \right) \quad (1.a.3)$$

Correzione (a cura del docente):

- (b) Studiare la stabilità di un punto di equilibrio, $(\bar{x}_1, \bar{x}_2)$, scelto a piacere tra quelli determinati al passo precedente, al variare di tutti i parametri. A tal scopo, al fine di determinare se il punto è stabile, è possibile usare il metodo indiretto di Lyapunov ovvero l'analisi delle soluzioni del modello lineare approssimato; qualora il punto di equilibrio risulti asintoticamente stabile, si indichi se tale proprietà è locale o globale; se la stabilità è solo locale, si fornisca una stima della RAS del punto di equilibrio.

Soluzione:

La stabilità locale di un sistema non lineare si può valutare utilizzando il metodo indiretto di Lyapunov, linearizzando il sistema attorno al punto di equilibrio. Questo porta il sistema alla forma:

$$\dot{x} = Ax \quad (1.b.1)$$

dove A rappresenta la matrice Jacobiana del sistema, valutata nel punto di equilibrio. La Jacobiana è definita come:

$$A = \frac{\partial f(x)}{\partial x} \quad (1.b.2)$$

dove $f(x)$ descrive la dinamica non lineare. Gli autovalori della matrice A permettono di determinare la stabilità del punto di equilibrio. Se tutti gli autovalori hanno parte reale negativa, il punto è asintoticamente stabile; se almeno uno ha parte reale positiva, il punto è instabile; se almeno uno è nullo allora non si può sapere se è instabile o se è stabile.

Linearizzando il sistema si ottiene:

$$A = \begin{pmatrix} 3x_1^2 + x_2^2 - 4 - 3x_1^2 & 2x_1x_2 - 1 \\ 2x_1x_2 + 1 & x_1^2 + 3x_2^2 - 4 \end{pmatrix}$$

Si va a valutare la matrice di linearizzazione nell'origine:

$$A_{P_1} = \begin{pmatrix} -4 & -1 \\ 1 & -4 \end{pmatrix}$$

Si può dimostrare che la locale asintotica stabilità è legata alla traccia (la somma dei termini della diagonale principale) e al determinante della matrice di linearizzazione. Quando $\det(A) > 0$ e $\text{tr}(A) < 0$ il sistema è stabile, altrimenti è instabile. In questo caso:

$$\det(A) = 17 \quad (1.b.3)$$

$$\text{tr}(A) = -8 \quad (1.b.4)$$

Essendo le condizioni rispettate, per il metodo indiretto di Lyapunov il punto P_1 è localmente asintoticamente stabile.

Si vuole dare una stima della Regione di Asintotica Stabilità (R.A.S.). Per far ciò si applica il metodo diretto di Lyapunov, il quale consiste nel determinare una funzione detta *candidata di Lyapunov*, V , che sia g.d.p. (globalmente definita positiva o definita positiva), e la cui derivata $\dot{V}$ sia almeno semi-definita negativa nel punto di equilibrio. Nel caso in cui queste due condizioni siano rispettate, V viene elevata a funzione di Lyapunov e il punto di equilibrio è asintoticamente stabile. La stima della R.A.S. è data dall'intervallo in cui la funzione ($\dot{V}$) si mantiene almeno semi-definita negativa, in quanto come già detto, indica che il sistema è asintoticamente stabile. Come funzione candidata si prende in esame una quadratica del tipo:

$$V = \frac{1}{2}x_1^2 + \frac{1}{2}x_2^2 \quad (1.b.5)$$

che è definita positiva ($V(x_1, x_2) > 0$ per $(x_1, x_2) \neq (0, 0)$) e nulla solo in $(0, 0)$.

Calcolandone la sua derivata, si ottiene:

$$\dot{V} = x_1\dot{x}_1 + x_2\dot{x}_2 \quad (1.b.6)$$

Sostituendo $\dot{x}_1$ e $\dot{x}_2$:

$$\dot{V} = x_1[x_1(x_1^2 + x_2^2 - 4) - x_2 - x_1^3] + x_2[x_2(x_1^2 + x_2^2 - 4) + x_1]$$

Semplificando:

$$\dot{V} = x_1^2(x_1^2 + x_2^2 - 4) - \cancel{x_1 x_2} - x_1^4 + \cancel{x_2 x_1} + x_2^2(x_1^2 + x_2^2 - 4)$$

Fattorizzando rispetto al termine $x_1^2 + x_2^2$:

$$\dot{V} = (x_1^2 + x_2^2)(x_1^2 + x_2^2 - 4) - x_1^4 \quad (1.b.7)$$

Per determinare quell'insieme di punti che tendono all'asintotica stabilità bisogna studiare il segno della funzione e valutare quando $\dot{V} \leq 0$.

Vicino all'origine, il termine $-x_1^4$ è trascurabile, per cui una prima stima della RAS è data da tutti quei punti che cadono nella circonferenza $x_1^2 + x_2^2 \leq 4$, in quanto, quando questa condizione è soddisfatta la $\dot{V}$ è definita negativa.

Si può comunque espandere la trattazione della RAS considerando anche $-x_1^4$ all'allontanarsi dal punto di equilibrio. La funzione si mantiene minore o uguale a zero sia quando $x_1^2 + x_2^2 \leq 4$ che quando $(x_1^2 + x_2^2)(x_1^2 + x_2^2 - 4) \leq x_1^4$. Di conseguenza la R.A.S. è rappresentata dall'unione tra i due intervalli. In termini matematici:

$$RAS = \{x_1^2 + x_2^2 \leq 4\} \cup \{(x_1^2 + x_2^2)(x_1^2 + x_2^2 - 4) \leq x_1^4\} \quad (1.b.8)$$

Correzione (a cura del docente):

- (c) Qualora il punto di equilibrio $(\bar{x}_1, \bar{x}_2)$ non sia GAS, si progetti un primo controllore, $x_3 = \varphi_1(x_1, x_2)$, che lo renda tale; altrimenti si assuma φ_1 identicamente nullo per i passi successivi.

Soluzione:

Il punto di equilibrio studiato $(0, 0)$ non è Globalmente Asintoticamente Stabile (GAS); di conseguenza si progetta un controllore $x_3 = \varphi_1(x_1, x_2)$ per renderlo tale.

Per determinare una legge di controllo che renda il punto di equilibrio globalmente asintoticamente stabile, si applica il teorema di Barbashin-Krasowskii. Il teorema dice che qualora l'origine fosse un punto di equilibrio per un sistema non lineare, questo è globalmente asintoticamente stabile se esistesse una funzione candidata di Lyapunov V globalmente definita positiva e radialmente illimitata la cui derivata, $\dot{V}$ è globalmente definita negativa.

Come funzione candidata di Lyapunov, si può scegliere una quadratica. Come visto nel punto precedente, questa è definita positiva ed è anche radialmente illimitata, in quanto allontanandosi dall'origine lungo qualsiasi direzione tende radialmente ad infinito:

$$V = \frac{1}{2}x_1^2 + \frac{1}{2}x_2^2$$

La sua derivata temporale lungo le traiettorie del sistema è:

$$\dot{V} = x_1\dot{x}_1 + x_2\dot{x}_2$$

Sostituendo le espressioni di $\dot{x}_1$ e $\dot{x}_2$ dal sistema dinamico, si ottiene:

$$\begin{aligned} \dot{V} &= x_1 [x_1 (x_1^2 + x_2^2 - 4) - x_2 - x_1^3] + x_2 [x_2 (x_1^2 + x_2^2 - 4) + x_3 + x_1] \\ \dot{V} &= x_1^2 (x_1^2 + x_2^2 - 4) - \cancel{x_1 x_2} + x_2^2 (x_1^2 + x_2^2 - 4) + x_2 x_3 + \cancel{x_1 x_2} - x_1^4 \\ \dot{V} &= (x_1^2 + x_2^2) (x_1^2 + x_2^2 - 4) + x_2 x_3 - x_1^4 \\ \dot{V} &= (x_1^2 + x_2^2)^2 - 4(x_1^2 + x_2^2) + x_2 x_3 - x_1^4 \\ \dot{V} &= \underbrace{2x_1^2 x_2^2 + x_2^4}_{A} - \underbrace{4(x_1^2 + x_2^2)}_{B} + \underbrace{x_2 x_3}_{C} - x_1^4 \end{aligned}$$

Per garantire che $\dot{V} \leq 0$, si vuole ottenere una $\dot{V}_d$ desiderata che sia definita negativa. Analizzando l'espressione della $\dot{V}$, si nota che il termine B è già definito negativo. Rimangono quindi il termine misto C e il termine definito positivo A . Si impone:

$$\dot{V} = \dot{V}_d = -k_1 x_2^2 \quad (1.c.1)$$

Il che porta ad imporre una legge di controllo $\varphi(x_1, x_2)$ pari a:

$$x_3 = \varphi_1(x_1, x_2) = -2x_1^2 x_2 - x_2^3 - k_1 x_2 \quad (1.c.2)$$

Sostituendo $\varphi_1(x_1, x_2)$ nella $\dot{V}$:

$$\dot{V} = -4(x_1^2 + x_2^2) - k_1 x_2^2 \quad (1.c.3)$$

Il termine $k_1 > 0$ e rappresenta il guadagno del controllore, gli altri termini sono invece definiti negativi, si ha quindi che la derivata della funzione candidata di Lyapunov è definita negativa, elevando $V(x_1, x_2)$ a funzione di Lyapunov.

Ciò dimostra, per Barbashin-Krasowskii, che l'origine è globalmente asintoticamente stabile (GAS).

Correzione (a cura del docente):

- (d) Realizzare una simulazione in ambiente Matlab/Simulink del sistema controllato e verificare il corretto funzionamento del controllore progettato. Riportare il codice Matlab delle funzioni realizzate, il diagramma Simulink, gli andamenti delle prime due variabili di stato e quello di x_3 .

Soluzione:

Per la simulazione su Simulink, il sistema di controllo è stato modellizzato come indicato dalla figura 1. In particolare, per una migliore gestione dei dati in Output, si è deciso di creare un sottosistema indicato in figura 2.

Di seguito sono riportati i codici dei blocchi funzione utilizzati per la Dinamica e il Controllo del modello completo:

```
function phi1 = controller(x) % Parametri
    %Guadagno del controllore
    k1=2;

    %unpack
    x1 = x(1);
    x2 = x(2);

    %Controllore
    phi1=-x2^3-2*x1^2*x2-k1*x2;
end
```

Listing 1: Definizione della funzione controller.

```
function dx = sistema_dinamico(x, phi1)
    % Ingressi del sistema
```

```

x_1 = x(1);
x_2 = x(2);
x_3 = phi1;

% Derivate
dx1 = x_1 * (x_1^2 + x_2^2 - 4) - x_2 - x_1^3;
dx2 = x_2 * (x_1^2 + x_2^2 - 4) + x_1 + x_3;

dx = [dx1; dx2];
end

```

Listing 2: Definizione della funzione `sistema_dinamico`.

Gli andamenti delle variabili di stato x_1 e x_2 sono mostrate in figura 3 e 4, mentre la legge di controllo è indicata in figura 5.

Correzione (a cura del docente):

- (e) Passando adesso a considerare anche la terza e quarta equazione del modello dinamico, si progetti un nuovo controllore che stabilizza l'intero sistema, applicando due volte la tecnica del backstepping. Più precisamente, si determini una prima legge di retroazione, $x_4 = \varphi_2(x_1, x_2, x_3)$, che usa la variabile x_4 per stabilizzare il sottosistema delle prime tre variabili di stato; poi, si determini una seconda legge di retroazione $u = \varphi(x)$ che stabilizzi l'intero sistema usando l'ingresso di controllo u .

Soluzione:

Il back-stepping è una tecnica di controllo non lineare, che consente di costruire una funzione di Lyapunov per stabilizzare l'intero sistema, supponendo di averne già stabilizzato una parte nascosta da una variabile d'ingresso. Dato quindi un sistema non lineare $\dot{x} = f(x, u)$ si progettano iterativamente le leggi di retroazione partendo da un sottosistema, stabilizzandolo, e proseguendo fino a stabilizzare l'intero sistema.

Si consideri quindi il sottosistema (x_1, x_2, x_3) :

$$\begin{cases} \dot{x}_1 = x_1(x_1^2 + x_2^2 - 4) - x_2 \\ \dot{x}_2 = x_2(x_1^2 + x_2^2 - 4) + x_1 + x_3 \\ \dot{x}_3 = x_1^2 - x_2^2 - x_4 \end{cases} \quad (1.e.1)$$

Per applicare la tecnica del Backstepping, deve essere possibile separare il vettore di stato in due insiemi di variabili:

$$x = \begin{pmatrix} \eta \\ \xi \end{pmatrix} \quad (1.e.2)$$

Questo vettore di stato deve essere strutturato affinché l'insieme di equazioni in η sia affine all'ingresso virtuale η e che l'insieme di equazioni in η e ξ sia affine all'ingresso reale u . In termini matematici si ottiene un sistema di equazioni di questo tipo:

$$\begin{cases} \dot{\eta} = f_1(\eta) + g_1(\eta)\xi \\ \dot{\xi} = f_2(\eta, \xi) + g_2(\eta, \xi)u \end{cases} \quad (1.e.3)$$

Applicando la teoria nel caso in esame, l'insieme di equazioni in η è costituito dal sistema in (1.a.1), già reso globalmente asintoticamente stabile nel punto precedente attraverso la legge di retroazione $\varphi_1(x_1, x_2)$, che va a sovrascrivere l'ingresso virtuale ξ , il quale è rappresentato da x_3 .

Detto questo, si prende come riferimento il sistema in (1.e.3) e si definisce una variabile z_1 data come la differenza tra l'ingresso virtuale e la legge in retroazione:

$$z_1 \triangleq \xi - \Phi(\eta) = x_3 - \varphi_1(x_1, x_2) \quad (1.e.4)$$

Derivando z_1 si ottiene una nuova dinamica che servirà successivamente per determinare il controllore φ_2 :

$$\dot{z}_1 = \dot{x}_3 - \dot{\varphi}_1 = (x_1^2 - x_2^2 - x_4) - \dot{\varphi}_1$$

Si applica il metodo diretto di Lyapunov considerando il sistema (1.e.1), scegliendo come candidata di Lyapunov una quadratica:

$$\dot{V}_1 = \dot{V} + z_1 \dot{z}_1 = \dot{V} + z_1(x_1^2 - x_2^2 - x_4 - \dot{\varphi}_1)$$

È certo che V sia una funzione di Lyapunov, non rimane quindi che porre $z_1 \dot{z}_1$ ad una $\dot{V}_{z_1}$ desiderata che sia definita negativa. Si impone quindi:

$$x_4 = \varphi_2(x_1, x_2, x_3) = k_2 z_1 + x_1^2 - x_2^2 + \dot{\varphi}_1 \quad (1.e.5)$$

Ottenendo una derivata della funzione candidata di Lyapunov pari a:

$$\dot{V}_1 = \dot{V} - k_2 z_1^2 \quad (1.e.6)$$

Con riferimento al sistema in (1.e.3), si è trovata quella funzione u che fa convergere a zero il vettore di stato η .

Riapplicando gli stessi passaggi per l'intero sistema:

$$\begin{cases} \dot{x}_1 &= x_1(x_1^2 + x_2^2 - 4) - x_2 - x_1^3 \\ \dot{x}_2 &= x_2(x_1^2 + x_2^2 - 4) + x_1 + x_3 \\ \dot{x}_3 &= x_1^2 - x_2^2 - x_4 \\ \dot{x}_4 &= -x_1^2 + 3x_4^2 + u(t) \end{cases} \quad (1.e.7)$$

Si prende come funzione candidata di Lyapunov nuovamente una quadratica. Derivandola si ottiene:

$$\dot{V}_2 = \dot{V}_1 + z_2 \dot{z}_2$$

Si determina che la legge che controlla l'intero sistema è:

$$u = \varphi(x_1, x_2, x_3, x_4) = -x_1^2 - 3x_4^2 + \dot{\varphi}_2 - k_3 z_2 \quad (1.e.8)$$

Dove z_2 è:

$$z_2 \triangleq x_4 - \varphi_2 \quad (1.e.9)$$

Procedendo con la sostituzione, si ha una funzione $\dot{V}_2$ definita negativa pari a:

$$\dot{V}_2 = \dot{V}_1 - k_3 z_2^2 \quad (1.e.10)$$

Correzione (a cura del docente):

- (f) Estendere la simulazione realizzata precedentemente per mostrare il corretto funzionamento dell'intero sistema controllato. Riportare il codice Matlab delle funzioni realizzate, il diagramma Simulink, gli andamenti delle variabili di stato e dei segnali φ_1 , φ_2 e φ .

Soluzione:

Per effettuare la simulazione, il sistema di controllo è stato modellizzato come indicato dalla figura 6. In particolare, per una migliore gestione dei dati in Output, si è deciso di creare i sottosistemi indicati in figura 8 e 9.

Di seguito sono riportati i codici dei blocchi funzione utilizzati per la Dinamica e il Controllo del modello completo:

```
function dx = sistema_dinamico(x, phi1)
    % Ingressi del sistema
    x_1 = x(1);
    x_2 = x(2);
    x_3=phi1;

    % Derivate
    dx1 = x_1*(x_1^2+x_2^2-4)-x_2-x_1^3;
    dx2= x_2*(x_2^2+x_2^2-4)+x_1+x_3;

    dx = [dx1; dx2];
end
```

Listing 3: Definizione della funzione sistema_dinamico.

```
function dx = sistema_dinamico2(x, phi_2)
    % Ingressi del sistema
    x_1 = x(1);
    x_2 = x(2);
    x_4 = phi_2;

    % Derivate
    dx3= x_1^2-x_2^2-x_4;

    dx = dx3;
end
```

Listing 4: Definizione della funzione sistema_dinamico2.

```
function dx = sistema_dinamico(x, phi, x_4)
    % Ingressi del sistema
    x_1 = x(1);
    u = phi;

    % Derivate
    dx4= -x_1^2+3*x_4^2+u;

    dx = dx4;
end
```

Listing 5: Definizione della funzione sistema_dinamico3.

```
function phi_1 = controller1(x) % Parametri
    k1 = 2; % Guadagno del controllore
```

```

%unpack
x_1 = x(1);
x_2 = x(2);

% Control Law
phi_1= -2*x_1^2*x_2-x_2^3-k1*x_2;
end

```

Listing 6: Definizione della funzione controller1.

```

function phi2 = controller2(x, dphi1, phi1, x_3) % Parametri
    k2 = 2;
    %unpack
    x_1 = x(1);
    x_2 = x(2);
    %definizione della variabile z1
    z1=(x_3-phi1);
    % Controllore
    phi2 =x_1^2+dphi1+k2*z1-x_2^2;

end

```

Listing 7: Definizione della funzione controller2.

```

function phi = controller3(x, dphi2, phi2, x_4) % Parametri
    k3 = 2;
    %unpack
    x_1 = x(1);
    %definizione della variabile z2
    z2 = x_4 - phi2;
    % Controllore
    phi = -k3*z2+dphi2-3*x_4^2-x_1^2;

end

```

Listing 8: Definizione della funzione controller3.

Gli andamenti delle variabili di stato x_1, x_2, x_3, x_4 sono mostrate nelle figure: 10, 11, 12 e 13; mentre le leggi di controllo sono indicate nelle figure: 14, 15 e 16.

Correzione (a cura del docente):

2 Parte Robotica

Una fabbrica impiega *bracci robotici* per inscatolare i prodotti provenienti da diverse linee di produzione, *muletti autonomi* per trasportare tali prodotti, e *nastri trasportatori* per traslare le scatole piene verso le linee di uscita. Precisamente, una sezione della fabbrica ospita un braccio articolato orizzontale, formato da due link di lunghezza 1.5 [m], ciascuno, e due giunti rotoidali; i prodotti arrivano da due linee di origine (LA e LB), che si trovano rispettivamente in posizione $(-10, 0)$ [m] e $(10, 0)$ [m]; i prodotti provenienti da LA devono essere trasportati da un primo muletto, M_1 , nella posizione di “pick1” $(-3, 0)$ [m], mentre quelli provenienti da LB devono essere traslati da un secondo muletto, M_2 , nella posizione di “pick2” in $(3, 0)$ [m]; il braccio robotico, alternativamente, prende un prodotto da una linea e lo inserisce in una scatola posizionata in posizione di “place” in $(0, 3)$ [m]; quando una scatola contiene prodotti da entrambe le linee, la scatola è considerata piena ed un nastro trasportatore la trasla verso una linea di uscita LC posizionata in $(0, 10)$ [m]. Questo meccanismo si ripete indefinitamente. Si chiede di progettare e simulare l'intero sistema sotto le seguenti ipotesi semplificative:

- la forza peso non ha nessun effetto sul braccio;
- la sincronizzazione tra il braccio e ciascun muletto avviene nel momento in cui la posizione dell'estremità del braccio e quella del muletto si trovano nella rispettiva posizione di “pick” a meno di una soglia di tolleranza di distanza di 0.010 [m];
- eventuali sovrapposizioni dei vari sottosistemi, che nella realtà genererebbero urti, si trascurano;
- il “buffer” delle posizioni di pick è nullo, perciò ciascun muletto, quando raggiunge la propria posizione di pick, aspetta il braccio robotico prima di tornare alla propria linea di origine;
- il nastro trasportatore con sopra il carico di una scatola si comporta come un sistema ad un polo dominante con funzione di trasferimento $G(s) = 1/(1 + s\tau)$, con $\tau = 0.9$ [s]; la traslazione si ferma non appena la scatola ha raggiunto LC;
- il braccio robotico aspetta che la scatola abbia raggiunto LC per partire con un nuovo pick and place;
- non ci sono requisiti sulla traiettoria del braccio nel movimento dalle posizioni di “pick” a quella di place, mentre è solo richiesto che l'estremità del braccio raggiunga tali punti;
- ciascun muletto è un unicycle.

L'organizzazione della soluzione, nonché la sua presentazione, sono lasciate libere agli studenti. Tuttavia, affinché essa siano ritenute corrette e complete, devono descrivere chiaramente almeno:

- il modello dinamico di ciascun componente coinvolto (braccio, muletti, nastro, ...) e la tipologia di operazione che deve svolgere (pick-and-place, point-to-point, trajectory tracking, path following, ...);
- uno o più opportuni controllori di basso livello per ciascun componente e per ciascuna operazione che deve svolgere, nonché la descrizione della procedura teoria seguita per il suo ottenimento;
- i pianificatori e gli automi per la sincronizzazione delle varie fasi e la sincronizzazione dei vari componenti (M_1 con il braccio, M_2 con il braccio, braccio con il nastro, ...);
- Il risultato di una o più simulazioni, con i rispettivi diagrammi Matlab/Simulink dei vari sottosistemi e i rispettivi codici Matlab, ciascuno chiuso in anello singolo con il proprio controllore; le simulazioni devono mostrare il raggiungimento dell'obiettivo, riportando almeno l'andamento dei segnali di interesse, di tutte le variabili di posizione, dei segnali di comando e di sincronizzazione.

Soluzione:

Per comprendere il funzionamento di tale sistema, si inizia definendo la struttura dei muletti, M_1 ed M_2 , che vengono implementati come **unicicli**. I due unicycli dovranno svolgere un movimento **P2P** (**Point-to-point**) che viene opportunamente sincronizzato con gli altri elementi del sistema. Il primo step è quello di definire il sistema di riferimento dell'unicycle rispetto al piano O_{xy} :

$$F_b = \left\{ p_b = \begin{pmatrix} x \\ y \end{pmatrix}, i_b = \begin{pmatrix} c_\psi \\ s_\psi \end{pmatrix}, j_b = \begin{pmatrix} -s_\psi \\ c_\psi \end{pmatrix} \right\} \quad (2.1)$$

Una volta definito il sistema di riferimento bisogna imporre il vincolo di non-traslazione laterale dell'uniciclo (vincolo cinematico), che si può esprimere mediante:

$$\hat{j}_b^T \cdot \dot{p}_b = (-s_\psi \quad c_\psi) \begin{pmatrix} \dot{x} \\ \dot{y} \end{pmatrix} = 0$$

Dato il modello cinematico dell'uniciclo

$$\dot{q} = \begin{pmatrix} \dot{x} \\ \dot{y} \\ \dot{\psi} \end{pmatrix}$$

è possibile esprimere il vincolo cinematico nella forma **Pfaffiana**, ovvero:

$$A(q) \cdot \dot{q} = (-s_\psi \quad c_\psi \quad 0) \cdot \begin{pmatrix} \dot{x} \\ \dot{y} \\ \dot{\psi} \end{pmatrix} = 0 \quad (2.2)$$

Dove $A(q)$ è un vettore in funzione della posizione e $\dot{q}$ è il vettore velocità. Si vogliono determinare tutte quelle velocità, ovvero quei vettori, che moltiplicati per $A(q)$ restituiscano 0, in modo tale da rispettare il vincolo cinematico. A livello geometrico ciò significa individuare tutti quei vettori presenti all'interno del **nucleo** (o **kernel**) di $A(q)$:

$$\dot{q} \in \ker \{A(q)\}$$

Un generico $\dot{q}$ appartenente al nucleo di $A(q)$ avrà una forma del tipo $\dot{q} = (a, b, c)^T$ e dunque è possibile sostituirlo nella forma Pfaffiana, ricavando:

$$\begin{cases} -as_\psi + bc_\psi = 0 \\ \forall a, c \in \mathbb{R} \end{cases} \rightarrow \begin{cases} b = at_\psi \\ \forall a, c \in \mathbb{R} \end{cases}$$

I due vettori che ci interessano e che soddisfano la relazione sono

$$\mu_1 = \begin{pmatrix} c_\psi \\ s_\psi \\ 0 \end{pmatrix}, \mu_2 = \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}$$

Questi due vettori, moltiplicati per $A(q)$, soddisfano la relazione e dunque fanno parte del nucleo:

$$\ker\{A(q)\} = \left\langle \begin{pmatrix} c_\psi \\ s_\psi \\ 0 \end{pmatrix}, \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix} \right\rangle$$

Il kernel rappresenta dunque una **base** per le velocità del sistema, quindi è possibile esprimerle come combinazione lineare della base per degli scalari v_1 e v_2 :

$$\dot{q} = v_1 \begin{pmatrix} c_\psi \\ s_\psi \\ 0 \end{pmatrix} + v_2 \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix} \rightarrow \begin{pmatrix} \dot{x} \\ \dot{y} \\ \dot{\psi} \end{pmatrix} = \underbrace{\begin{pmatrix} c_\psi & 0 \\ s_\psi & 0 \\ 0 & 1 \end{pmatrix}}_{S(q)} \begin{pmatrix} v_1 \\ v_2 \end{pmatrix}$$

Svolgendo i calcoli si otterrà il seguente sistema di equazioni:

$$\begin{cases} \dot{x} = v_1 c_\psi \\ \dot{y} = v_1 s_\psi \\ \dot{\psi} = v_2 \end{cases}$$

Ne consegue che:

- v_1 è uno scalare moltiplicato per il seno e il coseno dell'angolo e prende il nome di **VELOCITA' LONGITUDINALE** v ;

- v_2 è equiparato alla variazione angolare nel tempo, ovvero la velocità con cui ruota ψ (comando dello sterzo) e prende dunque il nome di **VELOCITA' ANGOLARE** ω .

E' possibile definire il vettore degli ingressi u :

$$u = \begin{pmatrix} v \\ \omega \end{pmatrix}$$

Integrandolo nel modello del sistema si ottiene quindi il **modello dinamico in coordinate cartesiane**:

$$\begin{cases} \dot{x} = v c_\psi \\ \dot{y} = v s_\psi \\ \dot{\psi} = \omega \end{cases} \quad (2.3)$$

Prima iniziare a descrivere il movimento P2P degli unicicli, è utile determinare le loro componenti in coordinate polari. Definito $P_i = (x_i, y_i)$, ovvero un generico waypoint sul piano, risultano:

$$\rho = \sqrt{(x - x_i)^2 + (y - y_i)^2}; \quad \varphi = \tan^{-1}\left(\frac{y - y_i}{x - x_i}\right); \quad \beta = \pi - (\psi - \varphi)$$

Le componenti trovate sono:

- Distanza ρ tra due waypoint (x_i, y_i) e (x, y) ;
- Orientamento dell'uniciclo β (angolo tra ρ e $\dot{q}$);
- Angolo φ di ρ rispetto all'asse orizzontale.

Si vuole determinare il modello dinamico del sistema in coordinate polari, cosa possibile derivando le 3 componenti trovate in precedenza:

$$\begin{cases} \dot{\rho} = -v c_\beta \\ \dot{\varphi} = \frac{v}{\rho} s_\beta \\ \dot{\beta} = \frac{v}{\rho} s_\beta - \omega \end{cases}$$

Il modello ottenuto è certamente corretto però per il task crea problemi in quanto vi è una singolarità in $\rho = 0$. La singolarità non è un assurdo, in quanto per $\rho = 0$ significherebbe che i due waypoint coincidono e non vi potrebbe essere alcun movimento P2P. Per ovviare a questo problema basti pensare che $\frac{v}{\rho}$ non è altro che una velocità media (velocità/distanza) e, di conseguenza, è possibile introdurre una nuova variabile $v^* = \frac{v}{\rho}$, utilizzabile imponendo $\rho \neq 0$. Il nuovo modello dinamico sarà dunque:

$$\begin{cases} \dot{\rho} = -\rho v^* c_\beta \\ \dot{\varphi} = v^* s_\beta \\ \dot{\beta} = v^* s_\beta - \omega \end{cases} \quad (2.4)$$

Per il teorema di Brockett i due sistemi hanno soluzioni coincidenti.

Passando alla progettazione del controllore dei muletti, l'idea di progetto è quella di far convergere a 0 la distanza tra i due waypoint. Consideriamo dunque una candidata di Lyapunov globalmente definita positiva per il task:

$$V = \frac{1}{2}\rho^2 + \frac{1}{2}\varphi^2 + \frac{1}{2}\beta^2 \rightarrow \dot{V} = -\rho^2 v^* c_\beta + \varphi v^* s_\beta + \beta v^* s_\beta - \beta \omega$$

Si vuole paragonarla ad una **derivata di Lie** progettuale desiderata:

$$\dot{V}_d = -k_\rho \rho^2 - k_\beta \beta^2$$

E' possibile eguagliare la $\dot{V}$ alla derivata di progetto un membro alla volta. Partendo con il primo termine, ovvero $-\rho^2 v^* c_\beta = -k_\rho \rho^2$, ci si accorge che vi è un problema per $\beta = \frac{\pi}{2}$, dove la $v^* \rightarrow \infty$. Si modifica dunque la derivata di progetto:

$$\dot{V}_d = -k_\rho \rho^2 c_\beta^2 - k_\beta \beta^2 \rightarrow v^* = k_\rho c_\beta \rightarrow v = \rho k_\rho c_\beta$$

Si consideri ora $\varphi v^* s_\beta + \beta v^* s_\beta - \beta \omega = -k_\beta \beta^2$: si può introdurre la funzione $\text{sinc}(\beta) = \frac{\sin(\beta)}{\beta}$, ottenendo:

$$\begin{aligned}\varphi v^* s_\beta + \beta v^* s_\beta - \beta \omega &= \varphi v^* \text{sinc}(\beta) \cdot \beta + \beta v^* s_\beta - \beta \omega \\ \varphi v^* \text{sinc}(\beta) \cdot \beta + \beta v^* s_\beta - \beta \omega &= -k_\beta \beta^2 \rightarrow \omega = \varphi v^* \text{sinc}(\beta) + v^* s_\beta + k_\beta \beta\end{aligned}$$

Infine, imponendo $\dot{V} = \dot{V}_d$, si ottiene:

$$\begin{cases} v = \rho k_\rho c_\beta \\ \omega = k_\beta \beta + k_\rho c_\beta (\varphi \text{sinc}(\beta) + s_\beta) \end{cases} \quad (2.5)$$

ovvero la legge di controllo di un singolo uniciclo. Ecco l'adattamento in MATLAB per entrambi i muletti:

```
function dStato = CinematicaM1(Stato, u)
    % Unpack
    x = Stato(1);
    y = Stato(2);
    psi = Stato(3);
    v = u(1);
    omega = u(2);

    % Dinamica del Muletto1
    dStato = [v * cos(psi);
              v * sin(psi);
              omega];
end
```

Listing 9: Definizione della funzione Dinamica Muletto A.

```
function u = ControllerM1(Stato, Ref)
    % Unpack
    x = Stato(1);
    y = Stato(2);
    psi = Stato(3);
    xi = Ref(1);
    yi = Ref(2);

    % Guadagni di controllo
    kr = 1.1;
    kb = 0.2;

    % Traslazione
    phi = atan2((y - yi), (x - xi));
    beta = pi - (psi - phi);
    rho = sqrt((x - xi)^2 + (y - yi)^2);

    % Legge di controllo
    v = rho * kr * cos(beta);
    omega = kb * beta + kr * cos(beta) * (phi * sinc(beta) + sin(beta));
    u = [v; omega];
end
```

Listing 10: Definizione della funzione Controllore Muletto 1.

```

function Ref = plannerM1(Stato, flag1, point)
    % Planner per il Muletto 1
    % Unpack
    x = Stato(1);
    y = Stato(2);

    % Tolleranza per considerare il punto raggiunto
    tol = 0.01;

    % Variabile persistente per l'indice del punto attuale
    persistent PosM1
    if isempty(PosM1)
        PosM1 = 1; % Inizializza al primo punto
    end

    % Coordinate del punto corrente
    xi = point(1, PosM1);
    yi = point(2, PosM1);

    % Calcolo della distanza dal punto corrente
    dist = sqrt((x - xi)^2 + (y - yi)^2);

    % Logica di switch tra i punti
    switch PosM1
        case 1
            if flag1
                PosM1 = 2; % Muletto in zona Pick uno
            end
        case 2
            if dist < tol
                PosM1 = 1; % Torna al primo punto
            end
    end
    Ref = point(:, PosM1);
end

```

Listing 11: Definizione della funzione Planner Muletto 1.

```

function dStato = CinematicaM2(Stato, u)
    % Unpack
    x = Stato(1);
    y = Stato(2);
    psi = Stato(3);
    v = u(1);
    omega = u(2);

    % Dinamica del muletto 2
    dStato = [v * cos(psi);
              v * sin(psi);
              omega];

```

```
end
```

Listing 12: Definizione della funzione Dinamica Muletto 2.

```
function u = ControllerM2(Stato, Ref)
    % Unpack
    x = Stato(1);
    y = Stato(2);
    psi = Stato(3);
    xi = Ref(1);
    yi = Ref(2);

    % Guadagni di controllo
    kr = 1.1;
    kb = 0.2;

    % Traslazione
    phi = atan2((y - yi), (x - xi));
    beta = pi - (psi - phi);
    rho = sqrt((x - xi)^2 + (y - yi)^2);

    % Legge di controllo
    v = rho * kr * cos(beta);
    omega = kb * beta + kr * cos(beta) * (phi * sinc(beta) +
        sin(beta));
    u = [v; omega];
end
```

Listing 13: Definizione della funzione Controllore Muletto 2.

```
function Ref = plannerM2(Stato, flag2, point)
    % Planner per il Muletto 2
    % Unpack
    x = Stato(1);
    y = Stato(2);

    % Tolleranza per considerare il punto raggiunto
    tol = 0.01;

    % Variabile persistente per l'indice del punto attuale
    persistent PosM2
    if isempty(PosM2)
        PosM2 = 1; % Inizializza al primo punto
    end

    % Coordinate del punto corrente
    xi = point(1, PosM2);
    yi = point(2, PosM2);

    % Calcolo della distanza dal punto corrente
    dist = sqrt((x - xi)^2 + (y - yi)^2);
```



```

% Logica di switch tra le posizioni raggiungibili dal
muletto
switch PosM2
    case 1
        if flag2
            PosM2 = 2; % Muletto in zona Pick2
        end
    case 2
        if dist < tol
            PosM2 = 1; % Torna al primo punto
        end
    end
end
Ref = point(:, PosM2);
end

```

Listing 14: Definizione della funzione Planner Muletto 2.

Gli andamenti degli stati dei due unicii e delle rispettive leggi di controllo sono mostrate da figura 25 a figura 28. Procedendo con la progettazione, si vuole sviluppare un braccio robotico orizzontale a due link con giunti rotoidali, parallelo al piano di lavoro. Esso dovrà compiere un movimento di **"Pick-and-place"**. Per determinarne il modello dinamico è necessario introdurre le grandezze in gioco:

- a_i è la lunghezza dell' i -esimo braccio;
- m_i è la massa dell' i -esimo braccio;
- p_i è il vettore posizione dell' i -esimo braccio;
- I_i è l'inerzia dell' i -esimo braccio nel centro di massa;
- l_i è la distanza tra un'estremità dell' i -esimo braccio e il suo centro di massa;
- q_i è l'angolo fra l' i -esimo braccio e il precedente (asse x nel caso del primo braccio);
- c_i , s_i sono coseno e seno degli angoli q_i corrispondenti.

Per la determinazione del modello dinamico del braccio si sfrutta l'equazione di Eulero-Lagrange:

$$\frac{d}{dt} \frac{\partial L}{\partial \dot{q}} - \frac{\partial L}{\partial q} = \underbrace{Q_{NC}}_{\text{Somma delle forze non conservative}} \quad (2.6)$$

Dove:

$$L = \underbrace{T}_{\text{Energia cinetica}} - \underbrace{U}_{\text{Energia potenziale}}$$

Considerando

$$q = \begin{pmatrix} q_1 \\ q_2 \end{pmatrix}; \quad Q_{NC} = \begin{pmatrix} \tau_1 \\ \tau_2 \end{pmatrix}; \quad U = 0 \rightarrow L = T$$

Si noti che $U = 0$ poichè il braccio è posto su un piano e di conseguenza gli effetti gravitazionali legati ai suoi movimenti sono trascurabili. Lo scopo sarà dunque determinare l'energia cinetica totale che, svolti i calcoli, risulta essere:

$$T = \frac{1}{2}(I_1^* + 2m_2 l_1 a_2 c_2) \dot{q}_1 + \frac{1}{2} I_2^* \dot{q}_2^2 + m_2 a_2 (a_2 + l_1 c_2) \dot{q}_1 \dot{q}_2 = L \quad (2.7)$$

Dove:

- $I_1^* = I_1 + m_1 a_1^2 + m_2 l_1^2 + m_2 a_2^2$;

- $I_2^* = I_2 + m_2 a_2^2$.

Svolgendo le derivate dell'equazione di Eulero-Lagrange e fattorizzando le equazioni si giunge alla seguente forma, detta Hamiltoniana:

$$B(q)\ddot{q} + h(q, \dot{q}) = \tau \quad (2.8)$$

che, espansa, risulta essere:

$$\begin{pmatrix} I_1^* + 2m_2 l_1 a_2 c_2 & m_2 a_2 (a_2 + l_1 c_2) \\ m_2 a_2 (a_2 + l_1 c_2) & I_2^* \end{pmatrix} \begin{pmatrix} \ddot{q}_1 \\ \ddot{q}_2 \end{pmatrix} + \begin{pmatrix} -m_2 l_1 a_2 s_2 (2\dot{q}_1 \dot{q}_2 + \dot{q}_2^2) \\ m_2 l_1 a_2 s_2 \dot{q}_1^2 \end{pmatrix} = \begin{pmatrix} \tau_1 \\ \tau_2 \end{pmatrix}$$

Ora si vuole determinare la legge di controllo del braccio robotico. Nello specifico, si vuole stabilizzare il sistema su un vettore desiderato $q_d = \begin{pmatrix} q_{d1} \\ q_{d2} \end{pmatrix}$, ovvero si vuole fare in modo che l'errore tenda a 0:

$$e \triangleq q - q_d \rightarrow 0$$

L'approccio iniziale è quello di provare a costruire un sistema del secondo ordine in cui l'errore converge a 0. Per prima cosa si scrive la dinamica desiderata:

$$\ddot{e} + k_v \dot{e} + k_p e = 0$$

Affinchè l'errore converga a zero, le costanti k_v e k_p devono essere definite positive, ovvero matrici simmetriche con tutti i minori principali maggiori di zero:

$$k_v = \begin{pmatrix} \bar{k}_v & 0 \\ 0 & \bar{k}_v \end{pmatrix}; \quad \begin{pmatrix} \bar{k}_p & 0 \\ 0 & \bar{k}_p \end{pmatrix}$$

Ora si vuole che la forma Hamiltoniana sia tale da far tendere l'errore a zero, vale a dire

$$B(q)\ddot{q} + h(q, \dot{q}) - \tau = B(q) \cdot (\ddot{e} + k_v \dot{e} + k_p e)$$

Si svolgono i conti, ricordando che $e = q - q_d \rightarrow \dot{e} = \dot{q} \rightarrow \ddot{e} = \ddot{q}$ per via del fatto che q_d viene considerato costante, ottenendo la seguente legge di controllo:

$$\tau = \underbrace{h(q, \dot{q})}_{\text{Compensazione non lineare}} - \underbrace{B(q) \cdot (k_v \dot{q} + k_p (q - q_d))}_{\text{Feedback puro}}$$

Chiamando $(k_v \dot{q} + k_p (q - q_d)) = PD$, cioè Proporzionale Derivativo, si ottiene l'equazione del **controllo a coppia calcolata**:

$$B(q) \cdot (\ddot{q} + PD) = 0 \quad (2.9)$$

Vengono riportate di seguito le implementazioni in MATLAB del braccio robotico:

```
function ddq = Manipolatore(q, dq, tau)
    % Unpack inputs
    q2 = q(2);
    dq1 = dq(1);
    dq2 = dq(2);

    % Parametri del sistema
    m1 = 1.5;
    m2 = 1.5;
    l1 = 1.5;
    l2 = 1.5;
    a1 = l1 / 2;
    a2 = l2 / 2;
    I1 = 0.2;
```

```

I2 = 0.01;

% Dinamica del sistema
p = m2 * a2 * (a2 * l1 * cos(q2));
I1s = I1 + m1 * a1^2 + m2 * l1^2 + m2 * a2^2;
I2s = I2 + m2 * a2^2;
B = [I1s + 2 * m2 * l1 * a2 * cos(q2), p;
     p, I2s];
h = [-2 * m2 * l1 * a2 * sin(q2) * dq1 * dq2 - m2 * l1 * a2
     * sin(q2) * dq1^2;
     m2 * l1 * a2 * sin(q2) * dq1^2];
ddq = B \ (tau - h);
end

```

Listing 15: Definizione della funzione Dinamica del Manipolatore.

```

function tau = ControllerBraccio(q, dq, q_d)
% Unpack
q2 = q(2);
dq1 = dq(1);
dq2 = dq(2);

% Parametri del sistema
m1 = 1.5;
m2 = 1.5;
l1 = 1.5;
l2 = 1.5;
a1 = l1 / 2;
a2 = l2 / 2;
I1 = 0.2;
I2 = 0.01;

% Dinamica del sistema
p = m2 * a2 * (a2 * l1 * cos(q2));
I1s = I1 + m1 * a1^2 + m2 * l1^2 + m2 * a2^2;
I2s = I2 + m2 * a2^2;
B = [I1s + 2 * m2 * l1 * a2 * cos(q2), p;
     p, I2s];
h = [-2 * m2 * l1 * a2 * sin(q2) * dq1 * dq2 - m2 * l1 * a2
     * sin(q2) * dq1^2;
     m2 * l1 * a2 * sin(q2) * dq1^2];

% Guadagno di controllo
lambda = 1;
Kp = lambda^2;
Kv = 2 * lambda;

% Legge di controllo
tau = h - B * (Kv * dq + Kp * (q - q_d));

```

```
end
```

Listing 16: Definizione della funzione Controller Manipolatore.

Si noti come i guadagni del controllore non sono espressi con valori casuali ma dipendono da un certo λ , ovvero dalle soluzioni del polinomio caratteristico della dinamica desiderata. Di seguito viene riportato il codice del planner per sincronizzare i movimenti del manipolatore con il resto del sistema:

```
function [theta, idx] = plannerBr(q, flag3, flag4, flag1, flag2, u)
    % Unpack
    q1 = q(1);

    % Variabile persistente per la posizione angolare del braccio
    persistent PosBr
    if isempty(PosBr)
        PosBr = 1;
    end

    tol = 0.01;
    ref = [u(PosBr); 0];
    theta = ref;
    idx = PosBr;

    switch PosBr
        case 1
            % Muovi il braccio dalla posizione di Pick1 alla posizione di Place1
            if flag1
                PosBr = 2;
            end
        case 2
            % Muovi il braccio dalla posizione di Place1 alla posizione di Pick2
            if abs(q1 - ref(1)) < tol && flag3
                PosBr = 3;
            end
        case 3
            % Muovi il braccio dalla posizione di Pick2 alla posizione di Place2
            if flag2
                PosBr = 4;
            end
        case 4
            % Ricomincia il ciclo di lavoro
            if abs(q1 - ref(1)) < tol && flag4
                PosBr = 1;
            end
        end
    end
end
```

Listing 17: Definizione della funzione Planner Manipolatore.

Gli andamenti dello stato del braccio e le rispettive leggi di controllo sono mostrate in figura 29 e figura 30. Per quanto concerne il nastro trasportatore, in questo macrosistema esso viene implementato come un sistema del primo ordine avente la seguente funzione di trasferimento:

$$G(s) = \frac{1}{(1 + s\tau)}; \tau = 0.9 \quad (2.10)$$

Il nastro trasporta la scatola verso LC e poi si arresta. Per scoprire la dinamica del nastro, bisogna antitrasformare la funzione di trasferimento che, attualmente, è espressa nel dominio di Laplace:

$$G(s) = \frac{Y(s)}{U(s)} \rightarrow Y(s) = G(s) \cdot U(s) = \frac{U(s)}{(1 + s\tau)} \rightarrow U(s) = Y(s) \cdot (1 + s\tau)$$

Adesso si sfruttano le proprietà della trasformata di Laplace, per le quali $sY(s) \Leftrightarrow \frac{dy(t)}{dt}$, dunque $\tau sY(s) \Leftrightarrow \tau \frac{dy(t)}{dt}$, il che porta ad avere:

$$U(s) = Y(s) \cdot (1 + s\tau) \Leftrightarrow y(t) + \tau \frac{dy(t)}{dt} = u(t)$$

Ritornando così nel dominio del tempo. Adesso si può effettuare un cambio di variabile ponendo $x(t) = y(t)$ risultando in:

$$\dot{x} = \frac{u - x}{\tau}$$

Si vuole che il sistema converga verso una determinata posizione desiderata x_d , dunque si applica una traslazione di questo tipo:

$$z = x - x_d \rightarrow \dot{z} = \dot{x}$$

Il che funziona perchè x_d è costante. La dinamica del sistema diventa:

$$\dot{z} = \frac{u - z - x_d}{\tau}$$

Volendo stabilizzare il sistema, si sceglie una candidata di Lyapunov del tipo

$$V = \frac{1}{2}z^2 = \frac{1}{2}(x - x_d)^2 \rightarrow \dot{V} = z\dot{z} = z \cdot \frac{u - z - x_d}{\tau} \quad (2.11)$$

La si impone pari ad una derivata di Lie progettuale

$$\dot{V}_d = -k_z z^2$$

Il tutto diventa

$$z \cdot \frac{u - z - x_d}{\tau} = -k_z z^2$$

Si ricava u :

$$u = x_d + z(1 - \tau k_z) \quad (2.12)$$

Che è la legge di controllo del nastro trasportatore, nella quale x_d è la posizione desiderata, ottenuta in ingresso al controllore da un planner e k_z è una costante positiva che definisce la velocità di convergenza del sistema. Di seguito viene riportata l'implementazione in MATLAB del nastro trasportatore:

```
function dx = NastroTrasportatore(x, u, x_d)
    % Parametro del sistema
    tau = 0.9;

    % Traslazione
    z = x - x_d;

    % Dinamica del sistema
    dx = (u - z - x_d) / tau;
end
```

Listing 18: Definizione della funzione NastroTrasportatore.

```

function u = ControllerN(x, x_d)
    % Guadagni di controllo
    tau = 0.9;
    kx2 = 0.8;

    % Traslazione
    z = x - x_d;

    % Legge di controllo
    u = x_d + z * (1 - tau * kx2);
end

```

Listing 19: Definizione della funzione ControllerN.

```

function [x_d, Contatore, PointIndex] = plannerN(x, flag1,
    flag2, flag3, flag4, N)
    tol = 0.01;

    persistent Npos
    if isempty(Npos)
        Npos = 2;
    end

    persistent Counter
    if isempty(Counter)
        Counter = 0;
    end

    % Determina il numero di pacchi dentro i nastri
    switch Counter
        case 0
            if flag1
                Counter = 1;
            end
        case 1
            if flag2
                Counter = 2;
            end
        case 2
            if flag4
                Counter = 0;
            end
    end

    % Determina lo switch di posizioni del nastro
    xNd = N(Npos);
    switch Npos
        case 1
            if abs(x - xNd) < tol
                Npos = 2;
            end
    end

```

```

        case 2
            if abs(x - xNd) < tol && flag3 && Counter == 2
                Npos = 1;
            end
        end

        x_d = xNd;
        Contatore = Counter;
        PointIndex = Npos;
    end
end

```

Listing 20: Definizione della funzione `plannerN`.

Gli andamenti dello stato del nastro trasportatore e la rispettiva legge di controllo sono mostrate in figura 31 e figura 32. Specificati i singoli sottosistemi robotici, si va ora alla progettazione dell'intera fabbrica. Si è deciso di adoperare un approccio *bottom-up*, ovvero scomporre il problema in più sottoproblemi da risolvere in modo separato.

In particolare, si utilizzano due sincronizzatori, uno dedicato alla gestione del braccio manipolatore in relazione ai muletti e l'altro alla sincronizzazione tra il braccio e il nastro trasportatore.

Il primo sincronizzatore coordina il braccio manipolatore con i due muletti, verificando che le operazioni di presa e rilascio dei pacchi avvengano solo quando entrambi i sistemi si trovano nelle posizioni corrette. Si ricevono come input le informazioni relative alla posizione del braccio manipolatore e dei muletti, e si producono due segnali, definiti `flagA` e `flagB`, che indicano rispettivamente quando il braccio è in posizione di prelievo (*Pick1*) rispetto al primo muletto e di prelievo (*Pick2*) rispetto al secondo muletto. Attraverso un controllo basato su una tolleranza predefinita, si verifica che il braccio e i muletti si trovino in condizioni tali da consentire l'operazione desiderata. Se la condizione è rispettata, il sincronizzatore abilita il flag corrispondente, altrimenti lo azzerà. In questo modo, si assicura che le operazioni dei muletti siano sincronizzate con i movimenti del braccio. Di seguito è riportata l'implementazione su MATLAB del primo sincronizzatore:

```

function [flagA, flagB] = Sincro1(StatoBr, StatoM2, StatoM1)
    % Sincronizzatore tra il braccio manipolatore e i muletti

    % Variabile persistente per i task del primo muletto
    persistent Task1
    if isempty(Task1)
        Task1 = 0;
    end

    % Variabile persistente per i task del secondo muletto
    persistent Task2
    if isempty(Task2)
        Task2 = 0;
    end

    % Tolleranza
    tol = 0.01;

    % Primo controllo: verifica se il braccio e il muletto 1
    % sono in posizione di Pick1
    if abs(StatoBr(1) - pi) < tol && abs(StatoM1(1) - (-3)) <
        tol

```

```

        Task1 = 1;
    else
        Task1 = 0;
    end
    flagA = Task1;

    % Secondo controllo: verifica se il braccio e il muletto 2
    % sono in posizione di Pick2
    if abs(StatoBr(1)) < tol && abs(StatoM2(1) - (3)) < tol
        Task2 = 1;
    else
        Task2 = 0;
    end
    flagB = Task2;
end

```

Listing 21: Definizione della funzione **Sincro1**.

Il secondo sincronizzatore gestisce invece il coordinamento tra il braccio manipolatore e il nastro trasportatore. Si ricevono come input lo stato del braccio e quello del nastro, e si producono due segnali di uscita, **flag3** e **flag4**. Il primo segnala gestisce il contatore di pacchi sul nastro, mentre il secondo indica che il braccio ha completato il rilascio di un pacco sul nastro. Anche in questo caso, si utilizzano condizioni di tolleranza per verificare che il braccio e il nastro siano allineati nelle posizioni richieste. Di seguito è riportato il codice MATLAB per l'implementazione del secondo sincronizzatore:

```

function [flag3, flag4] = Sincro2(StatoBr, StatoN)
    % Sincronizzatore tra il braccio manipolatore e il nastro
    % trasportatore

    % Variabile persistente per il contatore
    persistent Cont
    if isempty(Cont)
        Cont = 0;
    end

    % Variabile persistente per il task del nastro
    persistent TaskN
    if isempty(TaskN)
        TaskN = 0;
    end

    % Tolleranza
    tol = 0.01;

    % Condizione per il contatore: verifica se il braccio sia
    % in posizione di Pick
    % e il pacco sia pronto sul nastro
    if abs(StatoBr(1) - pi/2) < tol && abs(StatoN - 3) < tol
        Cont = 1;
    else
        Cont = 0;
    end
end

```



```

    flag3 = Cont;

    % Condizione per il movimento del nastro: verifica se il
    % braccio sia in posizione di Place e il nastro sia pronto
    % per ricevere il pacco
    if abs(StatoBr(1) - pi/2) < tol && abs(StatoN - 10) < tol
        TaskN = 1;
    else
        TaskN = 0;
    end
    flag4 = TaskN;
end

```

Listing 22: Definizione della funzione **Sincro2**.

Entrambi i sincronizzatori utilizzano variabili persistenti per mantenere lo stato del sistema durante le operazioni. La tolleranza definita nel sistema influenza significativamente la precisione della sincronizzazione: valori troppo ampi possono portare a falsi positivi, mentre valori troppo stretti possono ritardare le operazioni.

La rappresentazione grafica dei segnali di sincronizzazione del sistema sono mostrate in figura 33

Correzione (*a cura del docente*):

Figure e diagrammi



img/Controllo_Non_Lineare/Controllo/SchemaC.pdf

Figura 1: Sistema di controllo del modello dinamico in (1.a.1)



img/Controllo_Non_Lineare/Controllo/outC.pdf

Figura 2: Output del sistema di figura 1



img/Controllo_Non_Lineare/Controllo/AndamentoX1.pdf

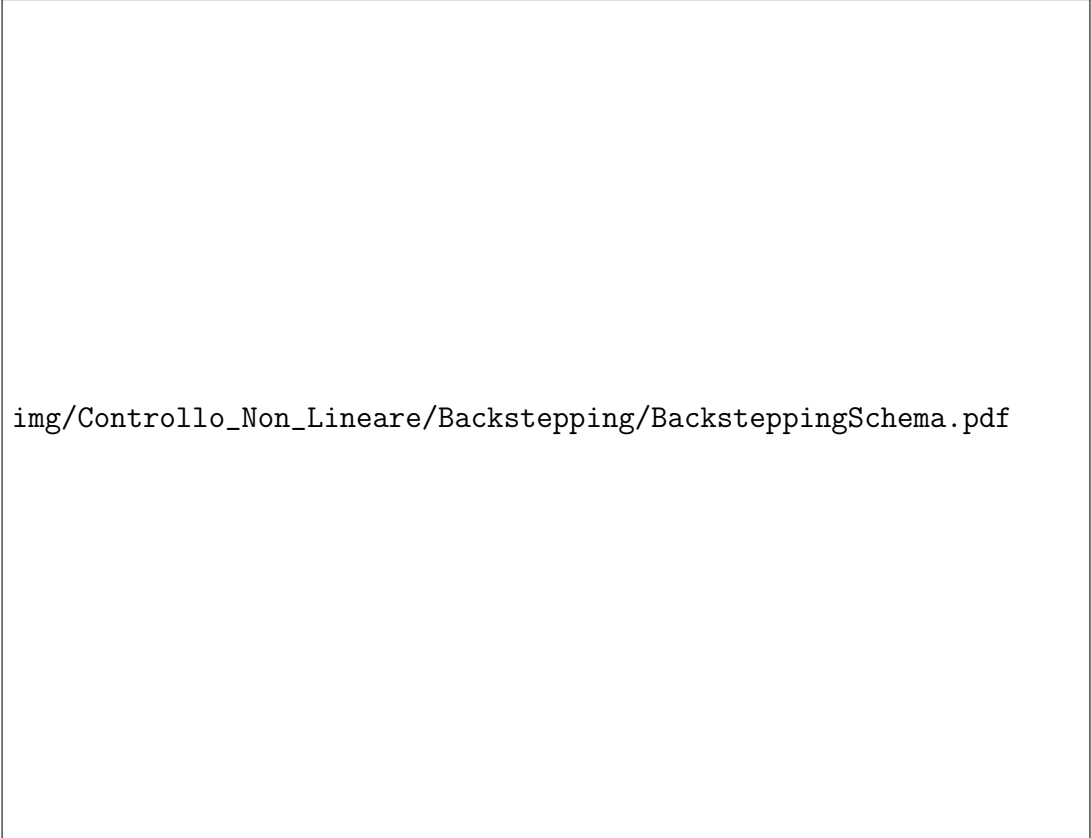
Figura 3: Andamento della variabile di stato x_1

img/Controllo_Non_Lineare/Controllo/AndamentoX2.pdf

Figura 4: Andamento della variabile di stato x_2

img/Controllo_Non_Lineare/Controllo/AndamentoPHI1.pdf

Figura 5: Andamento della variabile di controllo φ_1



img/Controllo_Non_Lineare/Backstepping/BacksteppingSchema.pdf

Figura 6: Schema a blocchi del doppio controllore in backstepping

img/Controllo_Non_Lineare/Backstepping/ControlloriBack.pdf

Figura 7: Controllori in backstepping



img/Controllo_Non_Lineare/Backstepping/OutControllo.pdf

Figura 8: Sottosistema che gestisce l'output dei blocchi di controllo



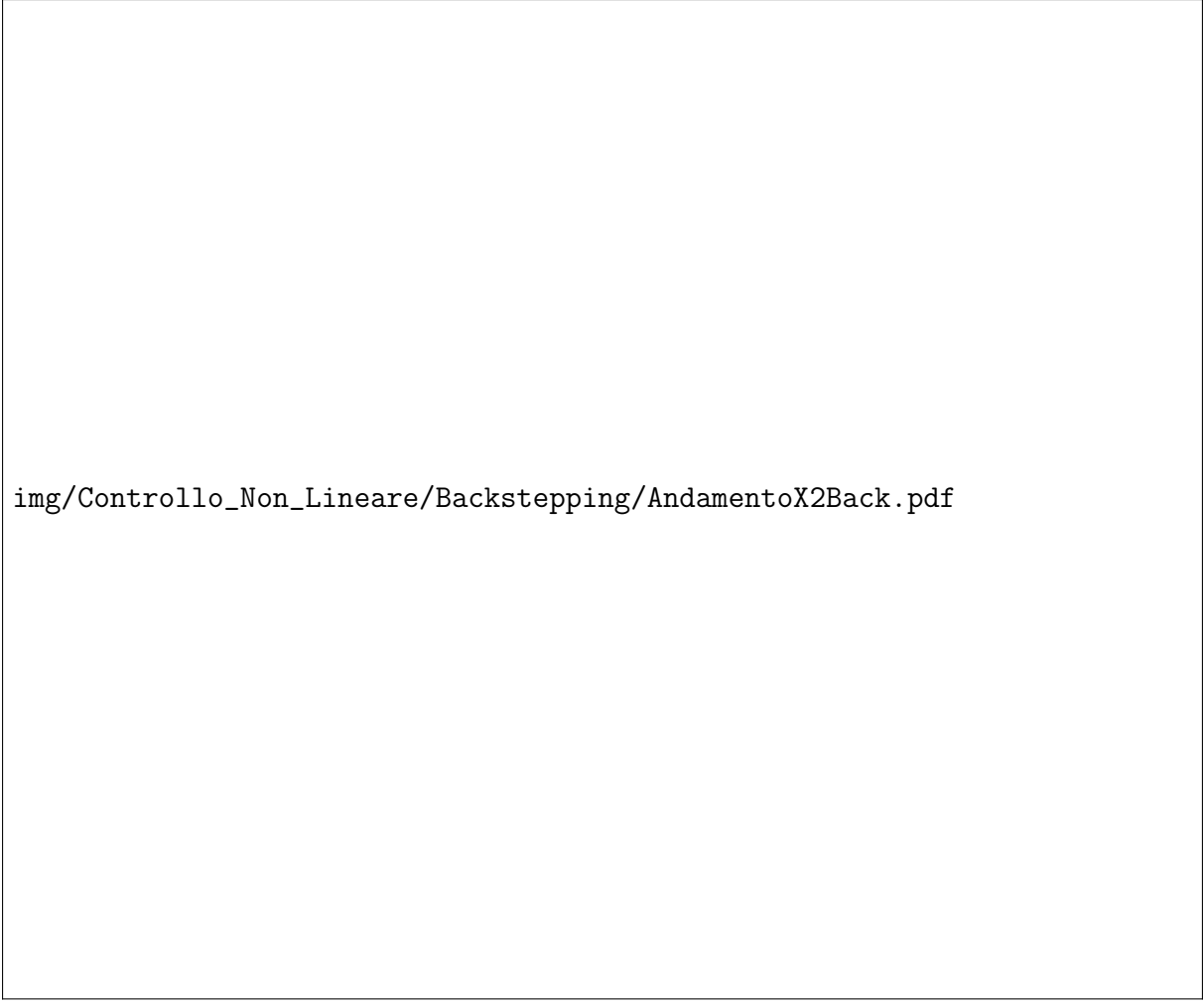
img/Controllo_Non_Lineare/Backstepping/OutX.pdf

Figura 9: Sottosistema che gestisce l'output dei blocchi di dei sottosistemi



img/Controllo_Non_Lineare/Backstepping/AndamentoX1Back.pdf

Figura 10: Andamento della variabile di stato x_1 del sistema di controllo in Backstepping



img/Controllo_Non_Lineare/Backstepping/AndamentoX2Back.pdf

Figura 11: Andamento della variabile di stato x_2 del sistema di controllo in Backstepping



img/Controllo_Non_Lineare/Backstepping/AndamentoX3Back.pdf

Figura 12: Andamento della variabile di stato x_3 del sistema di controllo in Backstepping



img/Controllo_Non_Lineare/Backstepping/AndamentoX4Back.pdf

Figura 13: Andamento della variabile di stato x_4 del sistema di controllo in Backstepping



img/Controllo_Non_Lineare/Backstepping/AndamentoPHI1Back.pdf

Figura 14: Andamento della variabile di controllo φ_1 del sistema di controllo in Backstepping



img/Controllo_Non_Lineare/Backstepping/AndamentoPHI2Back.pdf

Figura 15: Andamento della variabile di controllo φ_2 del sistema di controllo in Backstepping

img/Controllo_Non_Lineare/Backstepping/AndamentoPHIBack.pdf

Figura 16: Andamento della variabile di controllo φ del sistema di controllo in Backstepping

img/Robotica/Fabbrica.pdf

Figura 17: Schema a blocchi dell'intero sistema robotico

img/Robotica/Muletti/Muletti.pdf

Figura 18: Schemi del sistema "muletti" diviso in "muletto1" e "muletto2"

img/Robotica/Muletti/OutM.pdf

Figura 19: Schemi d'uscita dei due muletti

img/Robotica/Braccio/BraccioRobotico.pdf

Figura 20: Schema a blocchi del braccio manipolatore

img/Robotica/Braccio/OutBr.pdf

Figura 21: Schema d'uscita del braccio manipolatore

img/Robotica/Nastro_Trasportatore/Nastro_Trasportatore.pdf

Figura 22: Schema a blocchi del nastro trasportatore

img/Robotica/Nastro_Trasportatore/OutN.pdf

Figura 23: Schema d'uscita del nastro trasportatore

img/Robotica/Sincro.pdf

Figura 24: Schema del sistema di sincronizzazione

img/Robotica/Muletti/AndamentoM1.pdf

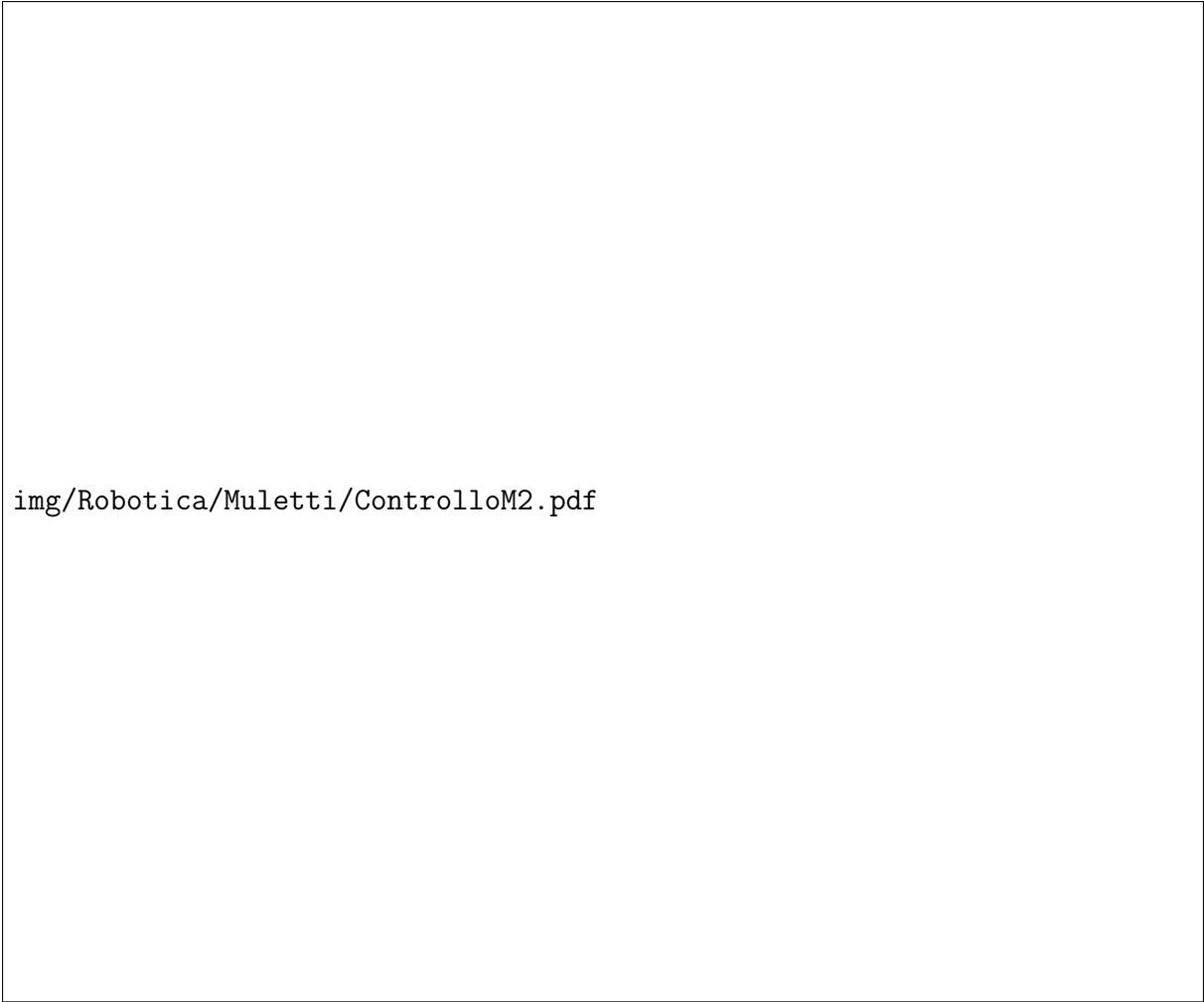
Figura 25: Andamento dello stato del primo muletto

img/Robotica/Muletti/ControlloM1.pdf

Figura 26: Andamento delle leggi di controllo del primo muletto

img/Robotica/Muletti/AndamentoM2.pdf

Figura 27: Andamento dello stato del secondo muletto



img/Robotica/Muletti/ControlloM2.pdf

Figura 28: Andamento delle leggi di controllo del secondo muletto

img/Robotica/Braccio/AndamentoBraccio.pdf

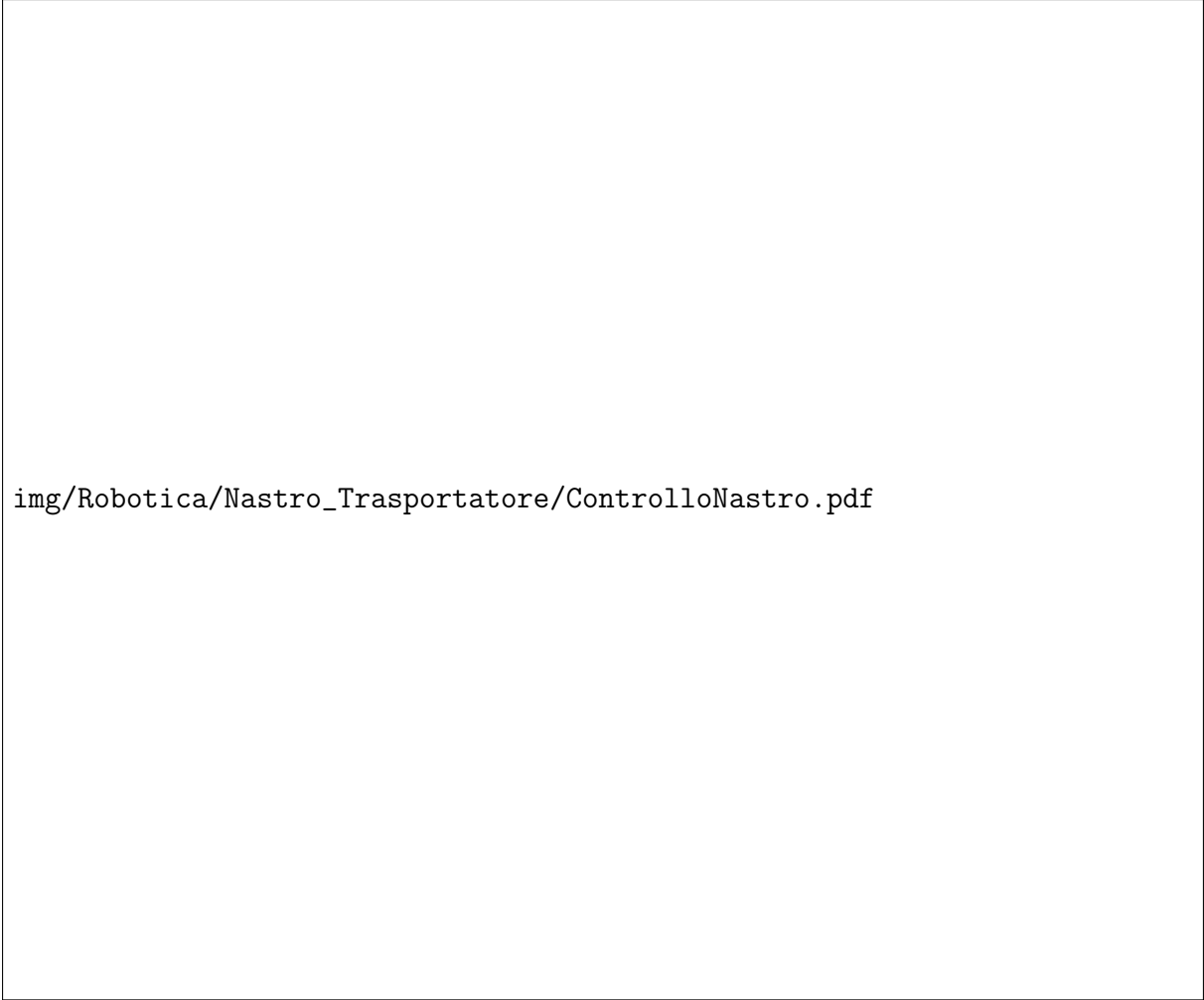
Figura 29: Andamento dello stato del Manipolatore

img/Robotica/Braccio/ControlloBraccio.pdf

Figura 30: Andamento delle leggi di controllo del manipolatore

img/Robotica/Nastro_Trasportatore/AndamentoNastro.pdf

Figura 31: Andamento dello stato del Nastro



img/Robotica/Nastro_Trasportatore/ControlloNastro.pdf

Figura 32: Andamento della legge di controllo del nastro trasportatore



img/Robotica/Sincronizzazione/Sincronizzazione.pdf

Figura 33: Segnali di sincronizzazione. I segnali **flag1** e **flag2** si riferiscono ai muletti, il segnale **flag4** si riferisce invece al nastro, mentre il segnale **idx** si riferisce al braccio robotico.

Fine dello svolgimento